Blockchain and Web3 Project Ideas

How to use this document

This is a revision of the original project list which I created based on my own knowledge of the domain (which is limited currently). I gave that list to Claude LLM to revise, and then I made a final non-technical edit of the document generated by Claude. The final document revised each project with concrete deliverables and an explicit statement of what you may not simply call a library for. It also added a few new project ideas.

Each project carries a type and a difficulty level. Implementation projects are judged on what you built; research projects are judged on the rigour of the evaluation. A project marked Hybrid needs both. Level 1 is accessible to anyone who has completed the course; level 3 assumes a willingness to read papers properly.

Read the "Beyond the library" line in every project before choosing it. It states the specific thing you must implement yourself. A submission that wires together existing tools without that component will not receive credit for implementation, however polished the demonstration. (This was added because I did not want students to simply call a library function and claim to have completed the work. The ones where libraries exist already may not be a publishable domain, or a major project. These are kept to provide training to those students who simply want to implement a known library functionality and extend it a little bit).

Evaluation criteria:

Every project below specifies deliverables and, for research projects, the evaluation standard.

Difficulty level of the projects:

I asked Claude to list the difficulty levels of the projects. I can't judge the implementation difficulties of the project myself; therefore exercise caution in believing these numbers.

Evaluation methodology is where challenge exists in most of these projects:

Several topics — liquidity management, oracle detection, MEV, ML-based detection — are easy to demo and hard to evaluate honestly. Backtests without impermanent loss, detectors without false-positive rates, and ML models evaluated on shuffled rather than temporal splits are the standard failure modes. This is flagged in each affected project.

Additional avenues to search for research problems:

Look at research papers published in recent Financial Crypto conferences, or eprint.iacr.org

Further, ACM CCS, Asia CCS, IEEE S&P, and USENIX Security are some of the other relevant conferences for this domain.

Part 1 — Project catalogue

Projects are grouped by area.

A. Bitcoin and UTXO-Chain Systems

Bitcoin is where the field's core ideas originate, its codebase is small enough to read, and its protocol surface is stable enough that a student project can engage with real internals rather than a moving target.

A1. A Bitcoin full-node validator from scratch (IBD subset)

Implementation · Difficulty 3/3

What to build. Write, in Rust or Go, a program that connects to the Bitcoin P2P network, performs the initial handshake, downloads headers, validates the proof-of-work chain, and fully validates a contiguous range of historical blocks — including script execution, signature verification, and UTXO-set maintenance. Sync from genesis to at least block 250,000 and assert that your UTXO set hash matches a reference.

Deliverables. Working node binary; UTXO-set commitment matching Bitcoin Core at chosen checkpoints; a written report on the three consensus subtleties you found hardest (e.g. BIP-30 duplicate coinbase, the OP_CHECKMULTISIG off-by-one, timestamp median rules).

Beyond the library. You may not use a Bitcoin library that implements script or consensus. Implement the script interpreter and UTXO management yourself; libsecp256k1 for ECDSA verification is permitted, as is a networking library for TCP.

References

- Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System (2008) https://bitcoin.org/bitcoin.pdf
- Bitcoin Developer Reference / BIPs repository https://github.com/bitcoin/bips
- Bitcoin Core source (consensus/ and script/ directories) https://github.com/bitcoin/bitcoin

A2. Taproot / Schnorr multisig and MuSig2 wallet

Implementation · Difficulty 3/3

What to build. Implement BIP-340 Schnorr signatures and the MuSig2 two-round key-aggregation protocol, then build a wallet that constructs, signs and broadcasts Taproot (P2TR) transactions on signet — both the key-path spend (aggregated, indistinguishable from single-sig) and a script-path spend revealing one leaf of a Taproot tree.

Deliverables. Library plus CLI wallet; signet transactions demonstrating both spend paths; measurement of on-chain byte cost versus an equivalent legacy P2SH multisig; a short security note on why MuSig2 needs two nonces.

Beyond the library. Implement MuSig2 nonce generation, aggregation and partial signing yourself from BIP-327. Do not call an existing musig2 crate. Getting nonce handling wrong is the entire point of the exercise — read the Wagner attack discussion.

References

- BIP-340: Schnorr Signatures for secp256k1 https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki
- BIP-341: Taproot https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki
- BIP-327: MuSig2 for BIP-340-compatible multi-signatures https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki
- Nick, Ruffing & Seurin, MuSig2: Simple Two-Round Schnorr Multi-Signatures, CRYPTO 2021 https://eprint.iacr.org/2020/1261

A3. Lightning Network payment-channel engine

Implementation · Difficulty 3/3

What to build. Implement the commitment-transaction state machine for a single Lightning channel: funding, commitment updates with revocation (the per-commitment secret chain), HTLC add/settle/fail, and unilateral close with penalty enforcement. Run it between two signet nodes.

Deliverables. Channel engine; a demonstrated penalty transaction that correctly punishes a party broadcasting a revoked state; analysis of the fee-inflation and pinning attack surface.

Beyond the library. Implement the revocation key derivation and commitment transaction construction from BOLT-3 directly. Do not wrap LND or Core Lightning — you may use them only as an interoperability target.

References

- Poon & Dryja, The Bitcoin Lightning Network (2016) https://lightning.network/lightning-network-paper.pdf
- BOLT specifications (especially BOLT-2 and BOLT-3) https://github.com/lightning/bolts

A4. Empirical study of Bitcoin fee-estimation and mempool dynamics

Research · Difficulty 2/3

What to build. Instrument several geographically separated Bitcoin nodes to log mempool arrivals and evictions over a sustained period. Compare Bitcoin Core's fee estimator against your own model (e.g. a queueing or survival-analysis model) on out-of-sample confirmation targets. Quantify how much nodes disagree about mempool contents and what that implies for fee estimation.

Deliverables. Dataset and collection harness; reproducible evaluation with a stated baseline; a short paper-style write-up with confidence intervals, not point estimates.

Beyond the library. This is a measurement project: the contribution is the methodology and the honest evaluation, not a tool. Temporal splits are mandatory — never evaluate a fee model on data that precedes its training window.

References

- Bitcoin Core fee estimation source (policy/fees.cpp) https://github.com/bitcoin/bitcoin/blob/master/src/policy/fees.cpp
- Möser & Böhme, Trends, Tips, Tolls: A Longitudinal Study of Bitcoin Transaction Fees, FC 2015 https://fc15.ifca.ai/preproceedings/bitcoin/paper_15.pdf

A5. Bitcoin script analyser and covenant explorer

Hybrid · Difficulty 2/3

What to build. Build a static analyser for Bitcoin Script that takes a scriptPubKey and determines its spending conditions symbolically. Then use it to survey the historical chain: how many outputs use non-standard scripts, and what would proposed covenant opcodes (CTV / BIP-119, and the OP_CAT revival discussion) enable that is currently impossible?

Deliverables. Analyser; a chain-wide survey with statistics; worked examples of at least three covenant constructions expressed in your analyser's notation.

Beyond the library. Write the Script interpreter and the symbolic engine yourself. The survey must run over real chain data, not a sample of textbook examples.

References

- BIP-119: CHECKTEMPLATEVERIFY https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki
- Bitcoin Script reference https://en.bitcoin.it/wiki/Script

B. Consensus, Staking and Node Infrastructure

B1. Single-Secret Leader Election prototype

Implementation · Difficulty 3/3

What to build. Implement an SSLE scheme end to end and measure it. Start from the shuffle-based construction that Ethereum's Whisk proposal uses: validators commit to trackers, proposers shuffle and re-randomise a candidate list, and ownership is proved in zero knowledge. Run it over a simulated validator set of at least 4,096 and measure shuffle cost, proof size and time-to-anonymity.

Deliverables. Working SSLE implementation; measurements of the anonymity-set growth rate versus shuffle rounds; an analysis of what an adversary controlling k% of shufflers learns.

Beyond the library. Implement the shuffle argument and the tracker re-randomisation. A wrapper around an existing ZK library that proves an unrelated statement does not count — the proof system must be doing the shuffle argument.

References

- Boneh, Eskandarian, Hanzlik & Greco, Single Secret Leader Election, ACM AFT 2020 https://eprint.iacr.org/2020/025
- Whisk: A practical shuffle-based SSLE protocol for Ethereum https://ethresear.ch/t/whisk-a-practical-shuffle-based-ssle-protocol-for-ethereum/11763
- Empirical security analysis of Whisk and homomorphic sortition under DoS and censorship (2025) https://arxiv.org/abs/2509.24955

B2. P2P eclipse-attack simulator and peer-scoring defence

Implementation · Difficulty 2/3

What to build. Build a discrete-event simulator of a Kademlia-style discovery network (Ethereum's discv5) with configurable node counts, churn and adversarial peers. Reproduce the Marcus–Heilman–Goldberg low-resource eclipse attack in simulation, then design and evaluate a peer-scoring or table-diversity defence against it.

Deliverables. Simulator; reproduction of the attack with success probability as a function of attacker IP count; your defence with a measured false-positive rate on honest churn.

Beyond the library. Implement the routing-table logic (bucket structure, eviction policy, endpoint verification) yourself — that is precisely where the vulnerability lives. Using libp2p as a transport is fine; using its peer-scoring module as your defence is not.

References

- Marcus, Heilman & Goldberg, Low-Resource Eclipse Attacks on Ethereum's P2P Network, ePrint 2018/236 https://eprint.iacr.org/2018/236
- Heilman, Kendler, Zohar & Goldberg, Eclipse Attacks on Bitcoin's P2P Network, USENIX Security 2015 https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/heilman
- Henningsen et al., Eclipsing Ethereum Peers with False Friends, EuroS&PW 2019 https://arxiv.org/abs/1908.10141
- Heimbach et al., Deanonymizing Ethereum Validators: The P2P Network Has a Privacy Issue, USENIX Security 2025 https://www.usenix.org/conference/usenixsecurity25

B3. Restaking and LSD cascade simulator

Research · Difficulty 2/3

What to build. Build an agent-based model of a restaking ecosystem: operators stake across multiple services, each with its own slashing conditions and correlation structure. Study when a single operator fault triggers a cascade. Calibrate against publicly available operator and stake-distribution data rather than invented numbers.

Deliverables. Simulation model with documented assumptions; sensitivity analysis over correlation and over-collateralisation parameters; a clear statement of which conclusions are robust and which are artifacts of the model.

Beyond the library. The intellectual work is modelling, not coding. You must state your assumptions explicitly and show sensitivity to each. A simulation whose conclusions depend on an unjustified parameter is worthless.

References

- EigenLayer whitepaper: The Restaking Collective https://docs.eigenfoundation.org/protocol/whitepaper
- Chitra & Pai, How much should you pay for restaking security? (2024) https://arxiv.org/abs/2408.00928
- Durvasula & Roughgarden, Robust Restaking Networks (2024) https://arxiv.org/abs/2407.21785

B4. Proof-of-stake attack replayer for Ethereum fork choice

Hybrid · Difficulty 3/3

What to build. Implement Ethereum's LMD-GHOST plus Casper FFG fork choice in isolation, then implement and measure known attacks against it — balancing attacks, ex-ante reorgs, and proposer-boost circumvention. Quantify how proposer boost changes the attacker's required stake.

Deliverables. Fork-choice implementation with a test harness; reproduction of at least two published attacks; measured stake thresholds versus the published analysis.

Beyond the library. Implement fork choice from the consensus specs. You may cross-check against a client, but the attack harness must drive your own implementation.

References

- Ethereum consensus specs (fork choice) https://github.com/ethereum/consensus-specs
- Neu, Tas & Tse, Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma, IEEE S&P 2021 https://arxiv.org/abs/2009.04987
- Schwarz-Schilling et al., Three Attacks on Proof-of-Stake Ethereum, FC 2022 https://eprint.iacr.org/2021/1413

C. MEV and Transaction Ordering

C1. Cross-domain MEV arbitrage simulator

Hybrid · Difficulty 3/3

What to build. Build an agent-based simulator of arbitrage across two or more domains with asynchronous state updates and differing finality. Model the searcher's inventory constraint and bridge latency explicitly — these, not price discovery, are what make cross-chain arbitrage hard. Validate against the empirical rates reported by Öz et al.

Deliverables. Simulator; a reproduction of at least one empirical finding from the literature; a sensitivity study over bridge latency and inventory.

Beyond the library. The simulator must model inventory and latency as first-class constraints. A price-difference scanner over two RPC endpoints is not this project.

References

- McMenamin, SoK: Cross-Domain MEV, arXiv 2308.04159 (2023) https://arxiv.org/abs/2308.04159
- Öz et al., Cross-Chain Arbitrage: The Next Frontier of MEV in DeFi, ACM SIGMETRICS/POMACS 9(3), 2025 https://doi.org/10.1145/3801489.3806898
- Obadia et al., Unity is Strength: A Formalization of Cross-Domain MEV, arXiv 2112.01472 https://arxiv.org/abs/2112.01472
- Qin, Zhou & Gervais, Quantifying Blockchain Extractable Value, IEEE S&P 2022 https://arxiv.org/abs/2101.05511

C2. Encrypted mempool with batched threshold decryption

Implementation · Difficulty 3/3

What to build. Implement a mempool in which transactions are encrypted to a committee key and decrypted only after ordering is fixed. Implement the threshold cryptosystem yourself, run it over a committee of at least 16

nodes, and measure the latency cost that threshold decryption adds per block. Then implement one of the published attacks against naive batched threshold encryption and show your design resists it.

Deliverables. Working prototype; end-to-end latency breakdown; demonstration of an attack against a naive variant and its mitigation in yours.

Beyond the library. Implement the threshold scheme (key generation, partial decryption, share combination). The project is about the cost and the failure modes, so the measurement harness matters as much as the protocol.

References

- Rondelet & Kilbourn, Mempool Privacy: An Economic Perspective, arXiv 2307.10878 (v1 titled "Threshold Encrypted Mempools") https://arxiv.org/abs/2307.10878
- Choudhuri et al., Mempool Privacy via Batched Threshold Encryption: Attacks and Defenses, USENIX Security 2024 https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri
- Bormet et al., BEAT-MEV: Epochless Approach to Batched Threshold Encryption, USENIX Security 2025 https://www.usenix.org/conference/usenixsecurity25/presentation/bormet
- Zhang et al., F3B: Per-Transaction Front-Running Protection, AFT 2023 https://doi.org/10.4230/LIPIcs.AFT.2023.3

C3. MEV redistribution via a backrun-refunding DEX router

Implementation · Difficulty 2/3

What to build. Build a router that detects when a user's swap creates an arbitrage opportunity, auctions the right to backrun it, and refunds a share of the proceeds to the original trader. Implement the auction and settlement on-chain and evaluate on forked mainnet state against historical swaps.

Deliverables. Contracts plus off-chain auctioneer; backtest over at least 1,000 historical swaps quantifying refund value; analysis of who bears the gas cost and whether the mechanism is incentive-compatible.

Beyond the library. The auction mechanism and settlement logic must be yours. Evaluation on forked historical state is required — a testnet demo with synthetic trades tells us nothing about whether it works.

References

- Flashbots MEV-Share documentation https://docs.flashbots.net/flashbots-mev-share/introduction
- Koegler, SoK: Current State of Ethereum's Enshrined Proposer-Builder Separation, arXiv 2506.18189 https://arxiv.org/abs/2506.18189
- Daian et al., Flash Boys 2.0, IEEE S&P 2020 https://arxiv.org/abs/1904.05234

C4. Sandwich-attack detector and defence evaluation over historical data

Research · Difficulty 2/3

What to build. Build a classifier that identifies sandwich attacks in historical Ethereum blocks from transaction traces alone, validate it against a hand-labelled ground-truth set, then measure how the rate changed around the adoption of private order flow. Report precision and recall, not just counts.

Deliverables. Labelled dataset and labelling protocol; detector with a proper evaluation; a time series with an honest discussion of confounders.

Beyond the library. The contribution is measurement quality. You must construct ground truth manually for a sample and report inter-rater agreement if more than one person labels.

References

- Qin, Zhou & Gervais, Quantifying Blockchain Extractable Value, IEEE S&P 2022 https://arxiv.org/abs/2101.05511
- Yang et al., SoK: MEV Countermeasures — Theory and Practice, arXiv 2212.05111 https://arxiv.org/abs/2212.05111

D. Zero-Knowledge Proofs and Privacy

The original tasks which I had shared with Claude leaned heavily on calling existing toolchains. The versions below were suggested by Claude and require implementing the circuits and, in one case, the proof system itself.

D1. Implement Groth16 verification from scratch and deploy it on-chain

Implementation · Difficulty 3/3

What to build. Implement pairing arithmetic over BN254 and a Groth16 verifier, first in Python or Rust for clarity, then as an EVM verifier contract. Take a circuit compiled by Circom, extract the verifying key, and verify real proofs. Measure your gas cost against the snarkjs-generated verifier and explain any difference.

Deliverables. Verifier implementation in a high-level language and in Solidity/Yul; gas comparison; a written explanation of the role of each of alpha, beta, gamma and delta in the verification equation.

Beyond the library. This is the reference example of 'more than calling a library'. You implement the pairing check; snarkjs is used only to generate proofs you verify and as a correctness oracle. An excellent extension is to demonstrate why gamma and delta must be independent by constructing a forged proof against a deliberately malformed key.

References

- Groth, On the Size of Pairing-Based Non-interactive Arguments, EUROCRYPT 2016 https://eprint.iacr.org/2016/260
- Gennaro, Gentry, Parno & Raykova, Quadratic Span Programs and Succinct NIZKs without PCPs, EUROCRYPT 2013 https://eprint.iacr.org/2012/215
- EIP-197: Precompiles for optimal ate pairing on alt_bn128 https://eips.ethereum.org/EIPS/eip-197
- Thaler, Proofs, Arguments, and Zero-Knowledge (textbook) https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf

D2. Compliant privacy pool with association sets

Implementation · Difficulty 3/3

What to build. Build a shielded pool in which a withdrawer proves membership of a chosen association set — a set of deposits they are willing to be publicly associated with — rather than merely proving non-membership of a blacklist. Write the Merkle-membership and nullifier circuits yourself in Circom or Halo2, and design the association-set update mechanism.

Deliverables. Circuits, contracts, and a client; an analysis of the anonymity set under realistic usage; a written discussion of what the scheme does and does not prove about the origin of funds.

Beyond the library. Do not clone a mixer tutorial. The novel content is the association-set design and its privacy analysis. Crucially, your nullifier must be a circuit-constrained public input — a Groth16 proof is re-randomisable, so using the proof bytes as the uniqueness key is an exploitable bug.

References

- Buterin et al., Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium, Blockchain: Research and Applications (2023) https://www.sciencedirect.com/science/article/pii/S2096720923000519
- Baghery, Kohlweiss, Siim & Volkhov, Another Look at Extraction and Randomization of Groth's zk-SNARK, FC 2021 https://eprint.iacr.org/2020/811
- Semaphore documentation https://semaphore.pse.dev/

D3. ZK proof of Web2 credential (zkTLS / zk-email style)

Implementation · Difficulty 3/3

What to build. Build a system that proves a statement about a signed Web2 artefact without revealing it — for example, proving that you hold a DKIM-signed email from a given domain containing a given claim, without revealing the email. Implement RSA verification and the regex/substring constraints inside the circuit.

Deliverables. Circuits and verifier; end-to-end demo on a real signed email; measurement of constraint count and proving time, with a discussion of what dominates.

Beyond the library. The circuit work — in-circuit RSA and string matching — is the project. Reason carefully about what is leaked by the public inputs you choose; that analysis is part of the deliverable.

References

- zk-email project and documentation https://prove.email/
- RFC 6376: DomainKeys Identified Mail (DKIM) Signatures https://www.rfc-editor.org/rfc/rfc6376

D4. Verifiable ML inference on-chain, done honestly

Hybrid · Difficulty 3/3

What to build. Take a small model (a decision tree or a two-layer network), express its inference as an arithmetic circuit, and verify the proof on-chain. Then do the part the original brief omits: measure and report the cost honestly — proving time, memory, constraint count and gas — as a function of model size, and identify the point at which the approach stops being practical.

Deliverables. Circuit and verifier; a scaling study across at least four model sizes; a clear statement of the practical ceiling and what dominates the cost.

Beyond the library. Using EZKL to convert a model is a single command and is not a project. The contribution here is the quantisation strategy, the circuit design, and a rigorous cost study. A negative result, properly measured, is a good outcome.

References

- Buterin, The promise and challenges of crypto + AI applications (2024) https://vitalik.eth.limo/general/2024/01/30/cryptoai.html
- awesome-zkml reading list https://github.com/worldcoin/awesome-zkml
- Kang et al., Scaling up Trustless DNN Inference with Zero-Knowledge Proofs (2022) https://arxiv.org/abs/2210.08674

D5. Benchmarking proof systems on a common circuit

Research · Difficulty 2/3

What to build. Define one non-trivial statement (for example, verifying 128 ECDSA signatures) and implement it in at least three proving stacks — Groth16 via Circom, a PLONK-family system, and a STARK-based system. Measure proving time, memory, proof size, verification cost and setup requirements under identical conditions.

Deliverables. Implementations in each stack; a controlled benchmark with hardware and version details; a discussion of which differences are fundamental to the proof system and which are implementation artefacts.

Beyond the library. The value is methodological rigour. Same statement, same hardware, reported variance across runs. Benchmarks that compare different statements are worthless and are the norm in this area.

References

- Gabizon, Williamson & Ciobotaru, PLONK, ePrint 2019/953 https://eprint.iacr.org/2019/953
- Ben-Sasson et al., Scalable, Transparent, and Post-Quantum Secure Computational Integrity (STARKs), ePrint 2018/046 https://eprint.iacr.org/2018/046
- Bowe, Grigg & Hopwood, Recursive Proof Composition without a Trusted Setup (Halo), ePrint 2019/1021 https://eprint.iacr.org/2019/1021

E. DeFi Protocols and Cryptoeconomic Security

E1. Uniswap v4 hook for volatility-aware liquidity

Implementation · Difficulty 2/3

What to build. Write a v4 hook that adjusts concentrated-liquidity ranges from realised volatility, and backtest it against historical swap data on forked state. Compare against a static range and a naive rebalancer, accounting properly for gas and for loss-versus-rebalancing.

Deliverables. Hook contract with tests; backtest harness over at least three months of real pool data; results including LVR, not just fee income.

Beyond the library. Reporting fee income without impermanent loss or LVR makes any strategy look profitable. The evaluation methodology is the deliverable; the hook is the easy part.

References

- Adams et al., Uniswap v4 Core whitepaper https://app.uniswap.org/whitepaper-v4.pdf
- Milionis, Moallemi, Roughgarden & Zhang, Automated Market Making and Loss-Versus-Rebalancing (2022) https://arxiv.org/abs/2208.06046
- Fan et al., Strategic Liquidity Provision in Uniswap v3, arXiv 2106.12033 https://arxiv.org/abs/2106.12033

E2. Oracle manipulation detector and circuit breaker

Implementation · Difficulty 2/3

What to build. Build a detector that monitors spot prices across venues against TWAP and flags manipulation. Validate it by replaying known historical oracle-manipulation incidents on forked state and measuring detection latency and false positives during ordinary volatility.

Deliverables. Detector plus on-chain circuit-breaker contract; replay of at least three documented incidents; a false-positive rate measured over a month of normal market data.

Beyond the library. A detector that fires during normal volatility is useless. Quantifying the false-positive rate on real data is the hard and necessary part.

References

- Zhou et al., SoK: Decentralized Finance (DeFi) Attacks, IEEE S&P 2023 https://arxiv.org/abs/2208.13035
- Wu et al., DeFiRanger: Detecting Price Manipulation Attacks on DeFi Applications https://arxiv.org/abs/2104.15068
- Chainlink Off-Chain Reporting protocol documentation https://docs.chain.link/architecture-overview/off-chain-reporting

E3. Lending-protocol liquidation simulator under stress

Research · Difficulty 2/3

What to build. Model a lending protocol's liquidation mechanism and study its behaviour under correlated price shocks and congested blockspace. Determine the conditions under which liquidations fail to clear and bad debt accumulates. Calibrate against real position data.

Deliverables. Simulation with documented calibration; identification of the parameter region where the protocol becomes insolvent; sensitivity analysis.

Beyond the library. Model gas competition among liquidators explicitly — liquidation failure under congestion is precisely the mechanism that creates bad debt, and models that ignore it get the answer wrong.

References

- Qin, Zhou, Gamito, Jovanovic & Gervais, An Empirical Study of DeFi Liquidations, IMC 2021 https://arxiv.org/abs/2106.06389
- Bartoletti, Chiang & Lafuente, SoK: Lending Pools in Decentralized Finance, FC 2021 Workshops https://arxiv.org/abs/2012.13230

E4. Stablecoin peg-stability mechanism analysis

Research · Difficulty 2/3

What to build. Compare the peg-defence mechanisms of at least three stablecoin designs (over-collateralised, algorithmic, and reserve-backed) using historical depeg episodes. Build a model of the reflexive dynamics and identify the conditions under which each design's stabilising feedback becomes destabilising.

Deliverables. Comparative model; empirical analysis of at least two real depeg events; a statement of the failure conditions for each design.

Beyond the library. Retrospective narrative is not analysis. Your model must make a falsifiable claim about the conditions for failure and be tested against data you did not use to build it.

References

– Klages-Mundt & Minca, While Stability Lasts: A Stochastic Model of Non-Custodial Stablecoins (2020) https://arxiv.org/abs/2004.01304

– Lyons & Viswanath-Natraj, What Keeps Stablecoins Stable?, Journal of International Money and Finance (2023) https://www.nber.org/papers/w27136

F. Layer 2, Rollups and Scalability

F1. Shared sequencer with cross-rollup atomicity

Implementation · Difficulty 3/3

What to build. Build a sequencer that accepts bundles spanning two rollups and guarantees all-or-nothing inclusion. Then engage with the hard part the literature identifies: atomic inclusion does not guarantee atomic execution, because each rollup's state transition can still revert independently. Implement a conditional-commitment scheme and show what it does and does not guarantee.

Deliverables. Sequencer; two minimal rollups; a demonstration of a cross-rollup swap succeeding and of the failure mode when one side reverts; a written analysis of the residual trust assumption.

Beyond the library. Scope this carefully — a full shared sequencer is a research programme. The rollups may be minimal. The contribution is the atomicity mechanism and an honest account of its limits.

References

– Gudgeon et al., SoK: Layer-Two Blockchain Protocols, ePrint 2019/360 https://eprint.iacr.org/2019/360

– Mamageishvili, Kelkar, Schlegel & Felten, Buying Time: Latency Racing vs Bidding in Fair Transaction Ordering / Atomic Execution is Not Enough, arXiv 2410.11552 https://arxiv.org/abs/2410.11552

– Motepalli, Freitas & Livshits, SoK: Decentralized Sequencers for Rollups, arXiv 2310.03616 https://arxiv.org/abs/2310.03616

F2. Rollup data-availability compression benchmark

Implementation · Difficulty 2/3

What to build. Collect real L2 batch data, implement several state-diff encodings (dictionary compression tuned to address and selector frequency, varint packing, domain-specific field encoding), and benchmark compression ratio against blob cost under EIP-4844 pricing. Include a naive general-purpose compressor as the baseline.

Deliverables. Compression implementations; benchmark over a substantial real dataset; a cost model converting compression ratio to actual blob fees.

Beyond the library. Running zstd over a file is the baseline you must beat, not the project. The contribution is a domain-specific encoding exploiting structure that a general compressor cannot see.

References

- EIP-4844: Shard Blob Transactions https://eips.ethereum.org/EIPS/eip-4844
- Hall-Andersen & Simkin, Foundations of Data Availability Sampling, CRYPTO 2024 https://eprint.iacr.org/2023/1079
- Al-Bassam, Sonnino & Buterin, Fraud and Data Availability Proofs (2018) https://arxiv.org/abs/1809.09044

F3. Recursive light client for instant sync

Implementation · Difficulty 3/3

What to build. Implement recursive proof aggregation over block headers so that a client can verify chain validity from a single succinct proof. Target a real chain's header format and measure the recursion overhead per aggregation step.

Deliverables. Recursive prover and verifier; measured proving time and proof size as a function of aggregated header count; a mobile or browser verification demo.

Beyond the library. The recursion circuit — verifying a proof inside a proof — is the work. Note that pairing-friendly curves do not compose freely; explaining your curve-cycle or curve-chain choice is part of the deliverable.

References

- Bowe, Grigg & Hopwood, Recursive Proof Composition without a Trusted Setup (Halo), ePrint 2019/1021 https://eprint.iacr.org/2019/1021
- Bonneau, Meckler, Rao & Shapiro, Mina (Coda) Protocol whitepaper https://minaprotocol.com/wp-content/uploads/technicalWhitepaper.pdf
- Ben-Sasson, Chiesa, Tromer & Virza, Scalable Zero Knowledge via Cycles of Elliptic Curves, CRYPTO 2014 https://eprint.iacr.org/2014/595

F4. Optimistic rollup fraud-proof game implementation

Implementation · Difficulty 3/3

What to build. Implement an interactive fraud-proof (bisection) game: two parties disagree about a state transition, bisect to a single instruction, and have an on-chain referee adjudicate that one step. Build the one-step executor for a small instruction set.

Deliverables. Bisection protocol and on-chain referee; a demonstrated successful challenge against a dishonest claim; analysis of the challenge period's relationship to censorship resistance.

Beyond the library. The one-step executor and the bisection game must be yours. This is the mechanism that makes optimistic rollups work and almost no student has implemented one.

References

- Kalodner et al., Arbitrum: Scalable, Private Smart Contracts, USENIX Security 2018 https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner
- Optimism Cannon fault-proof documentation https://docs.optimism.io/stack/protocol/fault-proofs/cannon

G. Smart Contract Security, Analysis and Tooling

G1. Cross-contract read-only reentrancy detector

Implementation · Difficulty 3/3

What to build. Build a detector for read-only reentrancy — where a view function returns stale state mid-reentrancy and a second protocol consumes it. Detection requires reasoning across DApp boundaries, so your tool must model interactions between contracts, not analyse one contract in isolation. Evaluate against known historical incidents.

Deliverables. Detector; evaluation on a labelled set including real incidents; precision and recall against a documented ground truth.

Beyond the library. Wrapping Slither's existing detectors is not the project. The contribution is cross-contract state-dependency analysis. Compare directly against SmartReco and explain where you differ.

References

- Zhang et al., SmartReco: Detecting Read-Only Reentrancy via Fine-Grained Cross-DApp Analysis, ICSE 2025 https://arxiv.org/abs/2409.18468
- Grieco et al., Echidna: Effective, Usable, and Fast Fuzzing for Smart Contracts, ISSTA 2020 https://agroce.github.io/issta20.pdf
- Ferreira Torres et al., The Eye of Horus: Spotting and Analyzing Attacks on Ethereum Smart Contracts, FC 2021 https://eprint.iacr.org/2021/284

G2. Invariant-driven fuzzing harness generator

Implementation · Difficulty 3/3

What to build. Build a tool that reads a contract and automatically generates stateful fuzzing invariants — conservation of supply, monotonicity of accumulators, access-control preconditions — and then drives Foundry or Echidna with them. Evaluate by measuring how many known bugs in a benchmark set the generated invariants catch versus hand-written ones.

Deliverables. Generator; evaluation on a public vulnerable-contract benchmark; analysis of which bug classes automated invariants systematically miss.

Beyond the library. Writing invariants by hand is an exercise, not a project. Automatic inference from contract structure is the contribution.

References

- Mossberg et al., Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts, ASE 2019 https://arxiv.org/abs/1907.03890
- Grieco et al., Echidna, ISSTA 2020 https://agroce.github.io/issta20.pdf
- Liu et al., Demystifying Invariant Effectiveness for Securing Smart Contracts, FSE 2024 https://arxiv.org/abs/2404.14580

G3. Exploit-simulation guard for pending transactions

Implementation · Difficulty 2/3

What to build. Build a monitor that simulates pending mempool transactions against current state in a local EVM, evaluates protocol invariants after simulation, and triggers a pause when an invariant would break. Evaluate by replaying historical exploits and measuring whether you would have caught them before inclusion.

Deliverables. Monitor plus guard contract; replay of at least three real exploits with timing analysis; a false-positive study over normal traffic.

Beyond the library. Consider the adversarial response explicitly: an attacker who knows you are watching the public mempool will use a private relay. Discuss what your defence is actually worth given that.

References

- Zhou et al., SoK: Decentralized Finance (DeFi) Attacks, IEEE S&P 2023 https://arxiv.org/abs/2208.13035
- Qin, Zhou & Gervais, On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi Protocols, IEEE S&P 2021 https://arxiv.org/abs/2103.02228
- Zhang et al., F3B, AFT 2023 https://doi.org/10.4230/LIPIcs.AFT.2023.3

G4. Differential testing of EVM implementations

Implementation · Difficulty 3/3

What to build. Build a fuzzer that generates random EVM bytecode and executes it against two or more independent EVM implementations (for example geth, revm, evmone, py-evm), comparing gas consumption, storage writes and revert behaviour. Any divergence is a potential consensus bug.

Deliverables. Bytecode generator and differential harness; a corpus of inputs achieving good opcode coverage; a report on any divergences found, including non-bugs and why they differ.

Beyond the library. The generator must produce semantically interesting programs, not random bytes — most random bytes revert immediately. Structure-aware generation is the core technical work.

References

- Ethereum execution specs and tests https://github.com/ethereum/execution-spec-tests
- Yang, Chen, Eide & Regehr, Finding and Understanding Bugs in C Compilers (Csmith), PLDI 2011 https://www.cs.utah.edu/~regehr/papers/pldi11-preprint.pdf

G5. LLM-assisted auditing: a rigorous evaluation

Research · Difficulty 2/3

What to build. Evaluate how well current language models find real smart-contract vulnerabilities. Build a benchmark from post-deployment incidents with known root causes, test several models with several prompting strategies, and report precision, recall and — critically — the false-positive burden an auditor would face.

Deliverables. Benchmark with a documented construction protocol; evaluation across models and prompts; an analysis of which vulnerability classes are found and which are systematically missed.

Beyond the library. Contamination control is essential: if the incidents predate the model's training cutoff, the result is meaningless. You must address this explicitly, ideally with contracts disclosed after the cutoff.

References

- Sun et al., GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis, ICSE 2024 https://arxiv.org/abs/2308.03314
- Liu et al., PropertyGPT: LLM-driven Formal Verification of Smart Contracts, NDSS 2025 https://arxiv.org/abs/2405.02580

H. Accounts, Wallets and Usability

Updated for EIP-7702, which shipped in Ethereum's Pectra upgrade in May 2025 and changes this area materially.

H1. secp256r1 (WebAuthn) verifier for smart accounts (revised from original #19)

Implementation · Difficulty 3/3

What to build. Implement secp256r1 signature verification in the EVM efficiently enough to be usable, and wire it into a smart account so that a passkey can authorise transactions. Benchmark your implementation against the RIP-7212 precompile where available and quantify the gap.

Deliverables. Verifier in Solidity or Yul with gas benchmarks; a working passkey-authorised account; analysis of which optimisations matter most.

Beyond the library. Field arithmetic and the scalar multiplication optimisations are the project. Calling an existing verifier library defeats the purpose.

References

- RIP-7212: Precompile for secp256r1 curve support https://github.com/ethereum/RIPs/blob/master/RIPS/rip-7212.md
- ERC-4337: Account Abstraction Using Alt Mempool https://eips.ethereum.org/EIPS/eip-4337
- W3C Web Authentication (WebAuthn) Level 3 https://www.w3.org/TR/webauthn-3/

H2. EIP-7702 delegation: security analysis and tooling

Hybrid · Difficulty 2/3

What to build. EIP-7702 lets an externally owned account temporarily delegate its code to a contract. Build tooling that analyses the resulting risk surface — what a malicious or buggy delegate can do to an EOA, how revocation works, and how delegation interacts with existing approvals — and implement a safe delegation manager.

Deliverables. Analysis of the attack surface with concrete exploit demonstrations on a testnet; a delegation manager contract with safety properties you state and test.

Beyond the library. This is genuinely new territory — EIP-7702 activated in May 2025 — so the security patterns are not yet settled. Original analysis is both possible and valuable here.

References

- EIP-7702: Set EOA account code https://eips.ethereum.org/EIPS/eip-7702
- Ethereum Pectra upgrade specification https://ethereum.org/en/roadmap/pectra/

H3. Intent-based transaction router with competing solvers (revised from original #20)

Implementation · Difficulty 3/3

What to build. Build an intent system end to end: a signed-preference format, a solver network competing to fill intents, a batch-auction settlement mechanism, and on-chain settlement. Then evaluate the mechanism — do competing solvers actually deliver better prices than direct execution, measured on historical data?

Deliverables. Intent format, at least two independent solver implementations, settlement contracts; an evaluation comparing achieved prices against direct AMM execution.

Beyond the library. The auction and settlement mechanism are the project, and the evaluation is what distinguishes it from a demo. Analyse solver collusion explicitly.

References

- CoW Protocol documentation (batch auctions) https://docs.cow.fi/
- UniswapX whitepaper https://app.uniswap.org/whitepaper-uniswapx.pdf
- Chitra et al., An Analysis of Intent-Based Markets (2024) https://arxiv.org/abs/2403.02525

H4. Social-recovery wallet with formally specified guardian policy

Implementation · Difficulty 2/3

What to build. Design and implement a recovery mechanism with time-locks, guardian thresholds and cancellation windows. Write a formal specification of the safety and liveness properties and verify them with a model checker (TLA+ or Alloy) before implementing.

Deliverables. Formal model with checked properties; contract implementation matching the model; a demonstration of an attack the model checker found.

Beyond the library. The specification must come first and must find at least one design flaw. Report the flaw — a model check that finds nothing usually means the properties were too weak.

References

- Buterin, Why we need wide adoption of social recovery wallets (2021) https://vitalik.eth.limo/general/2021/01/11/recovery.html
- Lamport, Specifying Systems (TLA+) https://lamport.azurewebsites.net/tla/book.html

I. Cross-Chain and Interoperability

I1. Atomic cross-chain swap between Bitcoin and an EVM chain

Implementation · Difficulty 3/3

What to build. Implement an HTLC-based atomic swap between Bitcoin signet and an EVM testnet, then analyse the failure modes honestly: what happens under chain reorganisation, what the free-option problem costs the counterparty, and how timeout parameters must be chosen relative to each chain's finality.

Deliverables. Working swap across both chains; a written analysis of the free-option problem with a quantitative estimate; timeout parameter derivation from finality assumptions.

Beyond the library. Construct the Bitcoin-side script and transactions yourself. The economic analysis of the free option is what raises this above a tutorial.

References

- Herlihy, Atomic Cross-Chain Swaps, PODC 2018 (arXiv 1801.09515) https://arxiv.org/abs/1801.09515
- Zamyatin et al., SoK: Communication Across Distributed Ledgers, FC 2021 https://eprint.iacr.org/2019/1128
- Xu, Ackerer & Dubovitskaya, A Game-Theoretic Analysis of Cross-Chain Atomic Swaps with HTLCs (2020) https://arxiv.org/abs/2011.11325

I2. Bridge security: a systematic post-mortem and taxonomy

Research · Difficulty 2/3

What to build. Build a taxonomy of cross-chain bridge failures grounded in the actual incident record. For each major bridge compromise, determine the root cause from on-chain evidence and disclosure reports, classify it, and identify which classes current designs still admit.

Deliverables. Incident database with primary-source evidence for each entry; taxonomy with clear classification criteria; an assessment of which failure classes remain open.

Beyond the library. Every classification must be supported by on-chain evidence or an official post-mortem, cited. Journalism is not a primary source.

References

- Zamyatin et al., SoK: Communication Across Distributed Ledgers, FC 2021 https://eprint.iacr.org/2019/1128
- Lee, Lee & Kim, SoK: Not Quite Water Under the Bridge — Review of Cross-Chain Bridge Hacks (2022) https://arxiv.org/abs/2210.16209

I3. Light-client bridge with on-chain header verification

Implementation · Difficulty 3/3

What to build. Build a trust-minimised bridge where the destination chain verifies the source chain's consensus directly — implement a sync-committee light client (for Ethereum) or SPV verification (for Bitcoin) as a contract on the destination chain. Measure the gas cost of maintaining the light client and discuss whether it is economically viable.

Deliverables. Light-client contract; relayer; gas-cost measurement over a sustained period; a comparison against the trust assumptions of a multisig bridge.

Beyond the library. Implement the consensus verification — signature aggregation and committee transitions — in the contract. This is the difference between a trust-minimised bridge and a multisig with extra steps.

References

- Ethereum Altair light client specifications https://github.com/ethereum/consensus-specs/tree/dev/specs/altair/light-client
- Bünz et al., FlyClient: Super-Light Clients for Cryptocurrencies, IEEE S&P 2020 https://eprint.iacr.org/2019/226
- Kiayias, Miller & Zindros, Non-Interactive Proofs of Proof-of-Work, FC 2020 https://eprint.iacr.org/2017/963

J. Measurement, Economics and Governance

These are research-focused projects suitable for students who prefer empirical or theoretical work to systems building. They are not soft options — the methodological bar is high.

J1. On-chain governance: measuring voting power concentration

Research · Difficulty 2/3

What to build. Measure actual decision-making power across several DAOs. Go beyond token-holding Gini coefficients: compute the Banzhaf or Shapley-Shubik power index over historical proposals, account for delegation, and determine how often the outcome was decided by a small number of addresses.

Deliverables. Dataset of proposals and votes; power-index computation; an analysis of the gap between nominal and effective decentralisation.

Beyond the library. Token distribution is a poor proxy for power. Computing a genuine power index over the delegation graph is the technical contribution.

References

– Fritsch, Müller & Wattenhofer, Analyzing Voting Power in Decentralized Governance (2022) https://arxiv.org/abs/2208.01636
– Messias et al., Understanding Blockchain Governance (2023) https://arxiv.org/abs/2304.02625

J2. Privacy erosion in account-based chains

Research · Difficulty 3/3

What to build. Quantify how much address-clustering and deanonymisation is possible on an account-based chain using timing, gas-price fingerprinting, funding patterns and interaction graphs. Evaluate against a ground-truth set you can validate (for example, addresses publicly self-identified via ENS or donation lists).

Deliverables. Clustering methodology; validation against ground truth with precision and recall; a discussion of countermeasures.

Beyond the library. Handle this responsibly: analyse only public data, publish no deanonymisation of specific individuals, and frame the contribution as a privacy risk assessment. Discuss the research ethics explicitly in your report.

References

– Victor, Address Clustering Heuristics for Ethereum, FC 2020 https://fc20.ifca.ai/preproceedings/31.pdf
– Meiklejohn et al., A Fistful of Bitcoins, IMC 2013 https://cseweb.ucsd.edu/~smeiklejohn/files/imc13.pdf

J3. Blockspace fee-market mechanism design

Research · Difficulty 3/3

What to build. Analyse EIP-1559 and its successors as a mechanism. Under what conditions is it incentive-compatible for users and for miners/proposers? Build a simulation testing the theoretical claims under bursty demand, and evaluate at least one alternative mechanism against the same conditions.

Deliverables. Formal statement of the properties being tested; simulation with results; identification of the conditions under which the mechanism's guarantees fail.

Beyond the library. This requires engaging with the mechanism-design literature properly. State your incentive-compatibility notion precisely and prove or empirically refute it.

References

– Roughgarden, Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559 (2020) https://arxiv.org/abs/2012.00854
– EIP-1559 specification https://eips.ethereum.org/EIPS/eip-1559
– Chung & Shi, Foundations of Transaction Fee Mechanism Design, SODA 2023 https://arxiv.org/abs/2111.03151

J4. Post-quantum readiness of deployed blockchains

Hybrid · Difficulty 3/3

What to build. Assess what a cryptographically relevant quantum computer would break in a deployed chain, and prototype a migration. Quantify how much value sits in addresses with exposed public keys (for Bitcoin: reused addresses and P2PK outputs), then implement a post-quantum signature scheme as an alternative authorisation path and measure the size and cost impact.

Deliverables. Chain-wide exposure analysis with real numbers; a prototype using ML-DSA (FIPS 204) or SLH-DSA (FIPS 205); a size and cost comparison against ECDSA/Schnorr.

Beyond the library. The exposure measurement over real chain data is the novel part; the signature scheme comes from a library, which is acceptable here because the contribution lies elsewhere. Be precise about the threat model — harvest-now-decrypt-later does not apply to signatures the way it does to encryption.

References

- NIST FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA) https://csrc.nist.gov/pubs/fips/204/final
- NIST FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA) https://csrc.nist.gov/pubs/fips/205/final
- Stewart et al., Committing to Quantum Resistance: A Slow Defence for Bitcoin (2018) https://royalsocietypublishing.org/doi/10.1098/rsos.180410

Part 2 — What counts as implementation

The distinction that matters is not how much code you write but whether you implemented the mechanism the project is about. Some guidance:

Permitted without question

Networking and serialisation libraries; elliptic-curve arithmetic primitives such as libsecp256k1 or arkworks field arithmetic; test frameworks; plotting and data-analysis libraries; an existing client used as a correctness oracle or interoperability target.

Permitted only if the mechanism is elsewhere

A proving library, when your contribution is the circuit rather than the proof system. A compression library, when it is your baseline rather than your result. A post-quantum signature implementation, when your contribution is the chain-wide exposure measurement.

Not permitted as the core of the project

Calling a library that performs exactly the task named in the title. If the project is "detect X" and your tool invokes an existing detector for X, there is no project. If the project is "verify a proof" and you call verifyProof(), likewise.

Always required

A written justification of every external dependency: what it does, why using it does not hollow out your contribution, and what you implemented instead. Half a page. This is the fastest way for you and for the instructor to establish that the project has substance.

Part 3 — Expected submission

Regardless of project type, the submission consists of four things. First, a repository that builds and runs from a clean checkout, with a README that states exactly how to reproduce every number in your report. Second, a report of roughly eight to twelve pages: problem, related work with correct citations, what you built, how you evaluated it, and what the limitations are. Third, the dependency justification described above. Fourth, a short demonstration.

Two things are marked heavily. A negative or partial result, honestly measured and clearly explained, scores well — the field has a serious problem with unreproducible claims and overstated benchmarks, and a student who reports an honest ceiling has learned more than one who reports a suspiciously clean success. Conversely, an evaluation that cannot support its claims will be marked down regardless of how much was built: a backtest without cost accounting, a detector without a false-positive rate, or a classifier evaluated on a shuffled split are all treated as unsupported.

Choose the topic before the tooling. The most common failure in this course is a student selecting a stack they want to learn and then searching for a problem to apply it to.